Ejercicio 1

El departamento de RR. HH. necesita **dar de alta empleados**, **exportarlos a JSON** y **volver a importarlos** para verificación.
Cada empleado tiene: dni, nombre, fechaAlta (ISO yyyy-MM-dd), salarioMensual (double) y activo (boolean).

Funcionalidades

1. **Dar de alta empleados:** el sistema pedirá datos repetidamente hasta que el usuario indique que no desea continuar.
2. **Exportar a JSON:** guarda la lista actual en data/empleados.json con **UTF-8** y **pretty printing**.
3. **Leer JSON:** muestra cuántos empleados contiene el fichero y lista **todos** los registros.
4. **Salir:** el programa finaliza solo cuando se elija esta opción.

Requisitos técnicos y de calidad

- **Java 17+** y **Maven**.
- Librería **Gson** con adaptador para LocalDate.
- **Modulariza** la solución en **clases separadas** y **paquetes**:
    - com.exercises → clase principal con el menú.
    - com.exercises.modelo → entidades de dominio.
    - com.exercises.util → utilidades (entrada por consola y adaptadores Gson).
- Buenas prácticas:
    - Encapsulación (atributos privados + getters/setters).
    - Path/Files y creación de directorios si no existen.
    - try-with-resources en operaciones de E/S.
    - Validación básica de entradas (no vacíos, números ≥ 0, fecha ISO).
    - Sin usar break/continue para salir de bucles.
    - Documentación interna y javadoc

Ejercicio 2

El departamento de **Ventas Online** necesita desarrollar una pequeña aplicación Java para **gestionar pedidos** y **exportarlos en formato JSON** para integrarlos con el sistema ERP de la empresa.

Cada pedido contiene los datos básicos de su cabecera (identificador, fecha y cliente) y un conjunto de **líneas de pedido** con la información de los productos adquiridos:

-codigo String

-descripcion String

- precioUnitario BigDecimal

-y unidades, entero

Tendrás que modelar las clases necesarias, así como los métodos necesarios.

El objetivo es crear un programa que permita:

- **Dar de alta pedidos** manualmente (pidiendo todos los datos por consola)
- **Exportar pedidos a JSON** con formato legible (pretty printing).
- **Leer un fichero JSON** existente y mostrar el total de pedidos junto con el detalle completo de cada uno.
- **Salir del programa** (el menú es un bucle repetir...hasta que el usuario elija salir).

Requisitos:

- Utilizar la librería **Jackson** para serializar y deserializar objetos en formato JSON.
- Comprender la configuración del **ObjectMapper** y el registro del módulo JavaTimeModule para fechas.
- Aplicar los tipos de datos adecuados (String, LocalDate, BigDecimal, List, int).
- Modularizar el código en paquetes y clases bien estructurados.

Anexo: BigDecimal

BigDecimal es una **clase de Java** que sirve para representar **números decimales con precisión exacta**, especialmente en **cálculos financieros o económicos**.

A diferencia de float o double, que son **tipos de coma flotante** y pueden tener errores de redondeo, BigDecimal **no pierde precisión** al sumar, restar o multiplicar valores con decimales.

```
double a = 0.1;
double b = 0.2;
System.out.println(a + b);
0.30000000000000004
```

Con BigDecimal:

```
import java.math.BigDecimal;

BigDecimal a = new BigDecimal("0.1");
BigDecimal b = new BigDecimal("0.2");
System.out.println(a.add(b));
0.3
```

Como BigDecimal es una clase (no un tipo primitivo), no puedes usar los operadores +, -, *, /. Tienes que usar **métodos**:

Operación	**Ejemplo**	**Resultado**
Suma	a.add(b)	suma exacta
Resta	a.subtract(b)	resta exacta
Multiplicación	a.multiply(b)	producto
División (con redondeo)	a.divide(b, 2, RoundingMode.HALF_UP)	divide con 2 decimales
Comparación	a.compareTo(b)	devuelve -1, 0 o 1